



CourseCube®
(Formerly Java Learning Center)

MicroServices

Study Guide

Module 8

Developing BookStore –V.1.5
(Hystix and Zuul)

Author
Srinivas Dande





CourseCube[®]
(Formerly Java Learning Center)

Spring Cloud Circuit Breaker

- ✓ **Spring Cloud Circuit Breaker** provides an abstraction across different circuit breaker implementations.
- ✓ It allows developer to choose the circuit breaker implementation that best fits your needs for your app.
- ✓ Supported Implementations
 - Netflix Hystrix
 - Resilience4J
 - Sentinel
 - Spring Retry

Exploring Netflix Hystrix

- ✓ MicroService based System consists of many MicroServices collaborating together.
- ✓ These MicroServices are prone to failure or delayed responses.
- ✓ If a MicroService fails it may impact on other MicroService affecting performance and possibly making other parts of application inaccessible or in the worst case bring down the whole application.
- ✓ The Hystrix library helps to control the interaction between services by providing
 - Fault Tolerance
 - Latency Tolerance.
- ✓ It improves overall resilience of the system by isolating the failing services and stopping the cascading effect of failures.
- ✓ Hysterix is built around using the Command design pattern.
- ✓ Each remote service call is wrapped in a HysterixCommand.
- ✓ Two Impotant features of Hystrix-
 - Fallback method
 - Circuit Breaker Implementation



Setup Hystrix in MicroServices

1) Add the following dependencies in pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
```

2) Add the @EnableCircuitBreaker the main Spring Application class.

```
package com.coursecube.bookstoreweb;
```

```
@SpringBootApplication
@EnableCircuitBreaker
public class BootMainApplication {
    ....
}
```

3) Specify the fallback method with @HystrixCommand

```
@GetMapping("/booksInfo")
@HystrixCommand(fallbackMethod = "myfallback")
public String getBooksInfo(){
    .....
    .....
    return info;
}

public String myfallback(){
    .....
    .....
    return info;
}
```

Exampe using Hystrix

Working Steps:

- 1) Take the Workspace: **Hystix-Demo**
- 2) Setup **Eureka Server** (Refer Previous Labs)
- 3) Create the MicroService – **BookSearchMS** (Code Given Below)
- 4) Create the MicroService – **BookStoreMS** (Code Given Below)
- 5) Start Eureka Server (Port : 8761)
- 6) Start 1 Instance of BookSearchMS (Port : 8000)
- 7) Start 1 Instance of BookStoreMS (Port : 5000)
- 8) Open the Browser and hit the following URL

<http://localhost:5000/mybooks>

<http://localhost:5000/authorBooks/Srinivas>

BookSearchMS – Files Required

1. BookSearchApplication.java	2. BookSearchController.java
3. application.properties	4. pom.xml

1. BookSearchApplication.java

```
package com.jlcindia.booksearch;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.circuitbreaker.EnableCircuitBreaker;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableEurekaClient
@EnableCircuitBreaker
public class BookSearchApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookSearchApplication.class, args);
    }
}
```



2. BookSearchController.java

```
package com.jlcindia.booksearch;

import java.util.ArrayList;
import java.util.List;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import com.netflix.hystrix.contrib.javanica.annotation.HystrixCommand;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@RestController
public class BookSearchController {

    @GetMapping("/allbooks")
    @HystrixCommand(fallbackMethod = "getAllBooksFallback")
    public List<String> getAllBooks() {
        System.out.println("---BookSearchController---getAllBooks()---");
        // Success Case
        List<String> booksList = new ArrayList<String>();
        booksList.add("Java");
        booksList.add("Spring");
        booksList.add("Spring Boot");
        booksList.add("Angular");
        booksList.add("React");
        return booksList;
    }

    @GetMapping("/booksByAuthor/{author}")
    @HystrixCommand(fallbackMethod = "getBooksByAuthorFallback")
    public List<String> getBooksByAuthor(@PathVariable String author) {
        System.out.println("---BookSearchController---getBooksByAuthor()---");

        // Problem Case
        if (1 == 1) {
            throw new ArithmeticException();
        }
        return null;
    }
}
```



```
public List<String> getAllBooksFallback() {
    System.out.println("---BookSearchController---getAllBooksFallback()---");
    List<String> booksList = new ArrayList<String>();
    booksList.add("No Books Available Currently");
    return booksList;
}

public List<String> getBooksByAuthorFallback(String author) {
    System.out.println("---BookSearchController---getBooksByAuthorFallback()--");
    List<String> booksList = new ArrayList<String>();
    booksList.add("No Books Available Currently for Author : "+author);
    return booksList;
}
}
```

3. application.properties

spring.application.name=**MyBookSearchMS**

server.port=**8000**

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

4. pom.xml

<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>**com.jlcindia.booksearch**</groupId>
<artifactId>**BookSearchMS**</artifactId>
<version>**1.0.SNAPSHOT**</version>
<packaging>jar</packaging>

<name>**MyBookSearchMS**</name>
<url>https://www.jlcindia.com</url>
<description>This is BookSeach MS developed by Srinivas Dande</description>



```
<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>
```



```
<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

BookStoreWeb – Files Required

1. BookStoreApplication.java	2. BookSearchProxy.java
3. BookStoreController.java	4. application.properties
5. pom.xml	

1. BookStoreApplication.java

```
package com.jlcindia.bookstoreweb;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.circuitbreaker.EnableCircuitBreaker;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
import org.springframework.cloud.openfeign.EnableFeignClients;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



```
@SpringBootApplication
@EnableFeignClients
@EnableEurekaClient
@EnableCircuitBreaker
public class BookStoreApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookStoreApplication.class, args);
    }
}
```

2. BookSearchProxy.java

```
package com.jlcindia.bookstoreweb;

import java.util.List;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@FeignClient("MyBookSearchMS")
public interface BookSearchProxy {
    @GetMapping("/allbooks")
    public List<String> getAllBooks();

    @GetMapping("/booksByAuthor/{authorName}")
    public List<String> getBooksByAuthor(@PathVariable String authorName);
}
```

3. BookStoreController.java

```
package com.jlcindia.bookstoreweb;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
```



```
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@RestController
public class BookStoreController {

    @Autowired
    BookSearchProxy bookSearchProxy;

    @GetMapping("/mybooks")
    public List<String> getMyBooks(){
        System.out.println("---BookStoreController---getMyBooks()---");
        List<String> booksList = bookSearchProxy.getAllBooks();
        return booksList;
    }

    @GetMapping("/mybooks/{author}")
    public List<String> getBooksByAuthor(@PathVariable String author) {
        System.out.println("---BookStoreController---getBooksByAuthor()---");

        List<String> booksList = bookSearchProxy.getBooksByAuthor(author);
        return booksList;
    }
}
```

4. application.properties

spring.application.name=**MyBookStoreWeb**

server.port=**5000**

eureka.client.service-url.defaultZone=http://localhost:8761/eureka

eureka.client.healthcheck.enabled=true

management.endpoints.web.exposure.include=*

management.endpoint.health.show-details=always



5. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.bookstoreweb</groupId>
<artifactId>BookStoreWeb</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>MyBookStoreWeb</name>
<url>https://www.jlcindia.com</url>
<description>This is BookStoreWeb developed by Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```